LocalShifts Operators Manual (v1)

This manual describes how to operate the current LocalShifts build, by role, and the exact steps for each activity.

Common URLs

- Public site: `/`
- Volunteer "My Signups": `/my`
- Admin:
  - Setup (first run only): `/admin/setup`
  - Login: `/admin/login`
  - Dashboard: `/admin/dashboard`
  - Users (Event Managers): `/admin/users`
  - Organizations: `/admin/organizations`
- Manager:
  - Login: `/manager/login`
  - Dashboard: `/manager/dashboard`
  - Events: `/manager/events`

Volunteer (Public)

View events

1. Open `/`.
2. Click an event card to view details and shifts.

Sign up for a shift

1. Open an event page (`/events/:slugOrId`).
2. Find a shift.
3. Click **Sign Me Up**.
4. Enter First name, Last name, Email.
5. Click **Confirm Signup**.
6. You should see a "You're signed up!" message.

Notes:

- If you are already signed up for the same shift with the same email, the signup will be rejected (duplicate protection).
- If the shift is full, signup will be rejected.
- If the event is cancelled, signup will be blocked.

See your upcoming signups (email-link flow)

1. Open `/my`.
2. Under **Email Me a Link**, enter your email and submit.

3. In development/test, the app shows a “Dev shortcut” link you can click.
4. In production, SMTP must be configured (otherwise you’ll get an “Email sending is not configured yet” message).

See your signups on a shared/public device (one-time view)

1. Open `/my`.
2. Request a link (or paste a link you received).
3. Choose the **One-time view** link (`remember=0`).
4. The page shows your upcoming signups without storing anything on the device.

Cancel a signup

1. Use the **Cancel** link shown on the event page for shifts you’re signed up for, or use a cancel link you received.
2. Open `/cancel/:token`.
3. Optionally add a note.
4. Click **Yes, Cancel My Signup**.

Event Manager

Log in

1. Open `/manager/login`.
2. Enter your email and password.
3. Click **Sign In**.

Create an event (draft)

Prerequisite: an Admin must create at least one Organization.

1. Open `/manager/events`.
2. Click **Create Event**.
3. Fill in Title, Organization, Date, and optional Description/Location fields.
4. Click **Create Draft**.

Edit an event

1. Open `/manager/events`.
2. Click the event.
3. Update fields.
4. Click **Save**.

Add a shift to an event

1. Open `/manager/events/:id/edit`.
2. Scroll to **Add Shift**.
3. Fill in role name, date, start time, duration, min/max.
4. Click **Add Shift**.

Edit a shift

1. Open `/manager/events/:id/edit`.
2. In **Shifts**, find the shift.
3. Click **Edit**.
4. Update fields.
5. Click **Save Shift**.

Notes:

- You cannot set `max volunteers` below the current number of active signups for that shift.
- Shifts cannot cross midnight.

Deactivate a shift (recommended over deletion if it has signups)

1. Open `/manager/events/:id/edit`.
2. In **Shifts**, click **Deactivate** on the shift.

Delete a shift

1. Open `/manager/events/:id/edit`.
2. In **Shifts**, click **Delete**.

Notes:

- Deletion is blocked if the shift has any signups (active or cancelled). Deactivate instead.

Publish / unpublish an event

1. Open `/manager/events/:id/edit`.
2. Click **Publish** (or **Unpublish**).

Notes:

- Publishing requires at least one active shift.

View signup roster for an event

1. Open `/manager/events/:id/edit`.
2. Click **View Signups**.
3. Or directly open `/manager/events/:id/signups`.

Add a manual signup (phone/in-person)

1. Open `/manager/events/:id/signups`.
2. Under the shift, open **Add Manual Signup**.
3. Enter name and email.
4. Click **Add**.

Notes:

- Manual signup respects capacity limits and duplicate protection.

- Manual signup is allowed even if the event is still a draft.

Remove a signup

1. Open `/manager/events/:id/signups`.
2. Click **Remove** next to an active signup.

Notes:

- This marks the signup as cancelled (it does not delete the record).

Export signups to CSV

1. Open `/manager/events/:id/signups`.
2. Click **Download CSV**.
3. Or open `/manager/events/:id/signups.csv`.

Cancel an event

1. Open `/manager/events/:id/edit`.
2. Click **Cancel Event**.

Notes:

- Current UI cancels with a default message ("Cancelled by organizer."). Custom messages are available via the ops endpoint (see below).
- Cancelling an event blocks new signups and triggers cancellation notifications (SMTP required for real delivery; dev logs to stdout).

Super Admin

First-time setup (create the first Super Admin)

Only available if no Super Admin exists yet.

Development/test:

1. Open `/admin/setup`.
2. Create the initial admin account.
3. Then use `/admin/login`.

Production:

1. Open `/admin/setup` with header `x-admin-token: $ADMIN_TOKEN`.
2. Create the initial admin account.

Log in

1. Open `/admin/login`.
2. Enter email and password.
3. Click **Sign In**.

Create an Organization

1. Open `/admin/organizations`.
2. Fill in name + slug (and optional color/contact email).
3. Click **Create**.

Notes:

- Organizations are required before managers can create events.

Create an Event Manager

1. Open `/admin/users`.
2. Fill in email, display name, and password.
3. Click **Create Manager**.

Activate/deactivate an Event Manager

1. Open `/admin/users`.
2. Click **Deactivate** or **Activate** on the manager.

Operator / Dev (Deployment, Maintenance, Troubleshooting)

Staging

- Staging base URL: `https://www.trtechapp.com`

Health checks

- `GET /healthz`: basic "process is up" check (public).
- `GET /ops/health`: checks DB connectivity (requires header `x-admin-token: $ADMIN_TOKEN`).
- `GET /ops/diag/schema`: checks for schema drift (missing tables/columns and migration visibility) (requires header `x-admin-token: $ADMIN_TOKEN`).

Reverse proxy notes (staging/production)

- Set `TRUST_PROXY=true` when running behind Apache/nginx so `req.ip` and rate limits work correctly.
- Ensure your reverse proxy overwrites `X-Forwarded-For` and `X-Forwarded-Proto` (don't trust client-supplied headers).
- Consider blocking `/ops/*` at the reverse proxy (allow only from your IP or localhost) even though it is token-protected.

Staging smoke test

Run a quick end-to-end check after deploys:

- `BASE_URL=https://www.trtechapp.com ADMIN_TOKEN=... sh scripts/smoke.sh`
- Optional email send: `EMAIL_TO=you@domain.com BASE_URL=... ADMIN_TOKEN=... sh scripts/smoke.sh`

SMTP throughput policy (PurelyMail default)

Goal: avoid provider throttling during reminder/broadcast bursts while still clearing scheduled sends quickly.

Policy values (production default):

- Reminder scheduler cadence: every 15 minutes.
- Reminder per-offset cap: `--limit-per-offset 100`.
- Effective reminder ceiling: if 3 offsets are active at once, max 300 sends per run (about 1,200/hour worst case).
- Manual broadcasts: treat as high-impact; run in off-peak windows for very large events.

Operational commands:

1. Dry-run estimate for next reminder sweep:
   - `node scripts/run.mjs reminder-scheduler --dry-run --limit-per-offset 100`
2. One live reminder sweep with cap:
   - `node scripts/run.mjs reminder-scheduler --limit-per-offset 100`
3. Loop mode for dedicated worker:
   - `node scripts/run.mjs reminder-scheduler --loop --interval-minutes 15 --limit-per-offset 100`

Escalation guidance:

1. If SMTP accepts are fast with no 4xx/5xx throttling, consider raising to `150`.
2. If throttling appears (timeouts, provider temp rejects), lower to `50` and retry next cycle.
3. Keep reminder sends below observed provider comfort level; do not increase cap during incidents.

Verification checklist for production sign-off:

1. Confirm current account-level send limits in PurelyMail dashboard and note date/time checked.
2. Run one dry-run and one live run with the chosen cap.
3. Confirm no SMTP throttle errors in app logs for two consecutive scheduler cycles.

Admin token in shell (ops convenience)

For operators who have access to the server repo folder, it can be convenient to have `ADMIN_TOKEN` in their shell environment for ops endpoints.

Recommended approach:

1. Copy `deploy/shell/codeSnippetForbashrc.example` to `codeSnippetForbashrc` in the repo root and lock it down:
   - `cp deploy/shell/codeSnippetForbashrc.example ./codeSnippetForbashrc`
   - `chmod 600 ./codeSnippetForbashrc`
2. Source it from `~/.bashrc`:
   - `echo 'test -f ~/projects/localshifts/codeSnippetForbashrc && . ~/projects/localshifts/codeSnippetForbashrc' >> ~/.bashrc`

Notes:

- `codeSnippetForbashrc` is ignored by git to avoid committing secrets.
- This snippet reads only `ADMIN_TOKEN` from `.env.staging` (it does not export DB credentials).

Application log file

Local dev compose writes application logs to `/app/logs/app.log` inside the app container (bind-mounted to `./logs/app.log` on the host).

View it:

1. `tail -f logs/app.log`
2. Or: `docker compose exec app sh -lc "tail -f /app/logs/app.log"`

Backup file permanence (staging)

When running JSON backup commands from inside the app container, backup files are written inside the container at `/app/backups`.

To make a backup file permanent on the host:

1. Verify files in the container:
   - `docker compose --env-file .env.staging -f docker-compose.staging.yml exec app ls -la /app/backups`
2. Copy the folder (or a single file) to host storage:
   - `docker cp localshifts-staging-app-1:/app/backups ./backups`

Notes:

- Replace `localshifts-staging-app-1` with your current app container name from `docker compose ... ps`.
- Container-local files can be lost when containers are recreated unless copied to host storage or a persistent volume.

Ops: template compile check

If a deploy fails with a Nunjucks template parse error (or you want to verify templates on a running container), call:

- `curl -fsS -H "x-admin-token: $ADMIN_TOKEN" "$APP_URL/ops/templates/compile"`

Ops: deployed version

To verify what's deployed (git SHA + build timestamp) without SSH:

- `curl -fsS -H "x-admin-token: $ADMIN_TOKEN" "$APP_URL/ops/version"`

Staging: deploy (managed Postgres + existing reverse proxy on the host)

Use this if your staging server already has nginx/Traefik/Apache/Caddy handling ports `80/443`.

1. On the server, create `.env.staging` based on `.env.staging.example`.
2. Set at minimum: `APP_URL`, `DATABASE_URL`, `SESSION_SECRET`, `ADMIN_TOKEN`.

3. Start the app container only:
    - `docker compose --env-file .env.staging -f docker-compose.staging.yml up --build -d`
4. Point your host reverse proxy upstream to `http://127.0.0.1:3000`.

Notes:

- If you start the `caddy` service while a host reverse proxy is already bound to `:80`/`:443`, you'll get a "port already in use" error. In that case, do **not** run the `caddy` profile.
- If you use a managed database (DigitalOcean, etc.), ensure the provider's firewall/trusted sources allow the server's public IP to connect.

Optional (staging): use the helper script to split "build" from "up" (and fail early if templates don't compile):

- `sh scripts/deploy-staging.sh`

Optional (staging): one-command update from git + deploy + smoke test:

- `GIT_REMOTE_URL=https://github.com/dmatthe4952/localshifts.git sh scripts/deploy-staging-from-git.sh`

Note: you do not need a modern Node.js installed on the host to deploy. The TypeScript build and template checks run inside the Docker image build (Node 20).

Staging: deploy (built-in Caddy HTTPS)

Use this if you do **not** have a reverse proxy already.

1. Create `.env.staging` based on `.env.staging.example` and set `CADDY_HOST` + `CADDY_EMAIL`.
2. Ensure DNS points to the server, and ports `80/443` are open.
3. Start app + Caddy:
    - `docker compose --env-file .env.staging -f docker-compose.staging.yml --profile caddy up --build -d`

Local dev: start

1. Run `docker compose up`.
2. Open `http://localhost:3000/`.

Local dev: seed demo data

1. Run `docker compose exec app npm run seed`.
2. The command prints demo credentials for `/admin/login` and `/manager/login`.

Staging: seed demo data

Allowed in `APP_ENV=staging` (blocked in `production`).

1. Run `docker compose --env-file .env.staging -f docker-compose.staging.yml exec app npm run seed`.
2. The command prints demo credentials for `/admin/login` and `/manager/login`.

Local dev: reset a user password

1. Run `docker compose exec app npm run set-password -- <email> <newPassword>`.

Run migrations

1. Run `docker compose exec app npm run migrate`.

Run integration tests (Docker)

1. Run `docker compose -f docker-compose.test.yml up --build --abort-on-container-exit`.

Cancel an event with a custom message (ops endpoint)

This is a temporary operational API until the manager UI supports custom messages.

1. Send:
    - `POST /ops/events/:slugOrId/cancel`
    - Header: `x-admin-token: $ADMIN_TOKEN`
    - Body JSON: `{"message":"..."}`.

Send a test email (ops endpoint)

Useful for verifying email delivery without creating a signup.

1. Send:
    - `POST /ops/email/test`
    - Header: `x-admin-token: $ADMIN_TOKEN`
    - Body JSON: `{"to":"you@example.com","subject":"Test","text":"Hello"}`.

SMTP / email behavior

- If SMTP is not configured, the app logs email contents to stdout (dev-friendly behavior).
- To send real email, set at minimum:
    - `SMTP_HOST`
    - `SMTP_FROM_EMAIL`
    - (Optionally) `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM_NAME`

Sendmail/postfix (host relay):

- If your staging server has a local MTA (sendmail/postfix), configure it to accept SMTP on `127.0.0.1` (commonly port `25`).
- Then set `SMTP_HOST=host.docker.internal`, `SMTP_PORT=25`, `SMTP_SECURE=false` in `.env.staging` and recreate the app container.

When to switch to an SMTP relay (smarthost):

- If you see repeated provider errors like `Service unavailable`, connection throttling, or long-lived deferrals in `mailq`, your droplet IP reputation is likely the issue.
- If outbound port `25` is blocked by your provider/network, direct delivery will fail.

- In those cases, configure a relay provider (SES/SendGrid/Mailgun/etc.) and either:
  - point LocalShifts directly at the provider via `SMTP_HOST/SMTP_PORT/SMTP_USER/SMTP_PASS`, or
  - configure sendmail to use a `SMART_HOST` and keep LocalShifts relaying to localhost.

Managed Postgres + SSL (DigitalOcean, etc.)

If you use a managed database and see TLS errors like `SELF_SIGNED_CERT_IN_CHAIN`, use one of these approaches:

- Easiest (encryption without certificate verification): set `sslmode=require` in `DATABASE_URL`.
- Strict (verify server cert/hostname): set `sslmode=verify-full` and provide the CA via `DATABASE_SSL_CA_PEM` or `DATABASE_SSL_CA_FILE`.

Staging: start on boot (systemd)

Optional helper unit lives in `deploy/systemd/localshifts.service`.

1. Copy it to `/etc/systemd/system/localshifts.service`.
2. Update `WorkingDirectory=` to your repo path.
3. Enable + start:
   - `sudo systemctl daemon-reload`
   - `sudo systemctl enable --now localshifts`

Email types currently sent:

- Signup confirmation (includes cancel link)
- My Signups sign-in link (prod: emailed; dev/test: shown on screen)
- Volunteer cancellation confirmation
- Cancellation alert to event manager
- Manager removal notice to volunteer

Current limitations (as of this manual)

- No full manager “cancel with custom message” UI yet (ops endpoint supports it).
- No reminder rules / scheduling (pg-boss) yet.
- No recurring events generation yet.
- No rich-text editor; descriptions are stored as escaped paragraphs (safe-by-default).